

Weekly Engineering Assignment 7 - Group 3 - S23

DNA Sequencing

```
In [203... # For printing basic information

def Sehyoun_Jang(prob):
    print("Name: Sehyoun Jang")
    print("sej324@lehigh.edu")
    print("Assignment ENGR10-S23-EN 7-3, section:", prob)
    print("GROUP 3")
    print("-----")
```

Perhaps the most important thing is how to read and organize txt files. It is not yet clear exactly what this assignment requires, so I need to look further

Section 1: DNA Sequencing

Step 1: Reading in the data

the file has the number on each line. Consider that, firstly, I will read the file and split it. And the number is always put on the first word. So, by slicing the the list, I can remove the number. and append it to empty string and return it

```
In [204... Sehyoun_Jang(1)

def DNA_read_function(filename):
    lst = []
    with open(filename, 'r') as file:
        temp = ""
        for line in file:
            a = line.rstrip().split()
            a = a[1:]
            for i in a:
                temp += i
        return temp # whole string

lst = (DNA_read_function("DNA_1.txt"))

print(lst)
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

```
-----
accgtagtgccggtgccctgagaacaggactgagtgatggcttccaactccagcgatgtgaaagaattcttagccaaagccaaa
gaagattttcttaaaaaatgggaaagtcccgcctcagaacacagcccacttgatcagtttgacgaatcaagaccctcggcacg
ggctccttcgggcggtgatgctggtgaaacacaaggagaccgggaaccactatgccatgaagatcctcgacaacagaaggtg
gtgaaactgaaacagatcgaacacacccatgaatgaaaagcgatcctgcaagctgtcaactttccgttcctcgtaaaactcgag
ttctccttcaaggacaactcaaacttatacatgggtcatggagtacgtgcccgggggagatgttctcacacctacggcggatc
ggaaggttcagttagcccatgccgtttctacgcggccagatcgtcctgacctttgagtatctgcactcgctggatctcatc
tacagggacctgaagccggagaatctgctcattgaccagcagggctacattcaggtgacagacttcggtttcgcaagcgctg
aagggccgcacttgacctgtgctggcaccctgagtacctggccctgagattatcctgagcaaaggctacaacaaggccgtg
gactggtgggcccctgggggttcttatctatgaaatggcgctggctaccgcccttcttcgagaccagccatccagatctat
gagaagatcgctctggaaggtgcgcttcccttccacttcagctctgacttgaaggacctgctgcggaacctcctgcaggtg
gatctaccaagcgcttgggaacctcaagaatgggtgcaacgatatcaagaaccacaagtgggttgcacaactgactggatt
gccatctaccagaggaaggtggaagctcccttcataccaaagtttaaggccctggggatagagtaactttgacgactatgag
gaagaagaaatccgggtctccatcaatgagaagtgtggaaggagtttctgagttttaggggcatgcctgtgcccccatgggt
tttcttttttcttttttcttttttgggtcggggggtgggagggttgattgaacagccagaggggcccagagttccttgcat
ctaatttcacccccacccaccctccatgggttagggggagcaggaagcccagataatcagaggggacagaaacaccagctgtcc
ccctcatccccctcacccctcctgccccctctccacttttcccttctcttccccacagccccccagccccctcagccctcca
gcccacttctgctgttttaaacgagtttctcaactccagtcagaccaggtcttgctggtgtatccagggaagggtatggaaa
gaggggctcacgcttaactccagccccacccacaccccatccacccaaccacaggccccacttgctaagggcaaatgaacg
aagcgcaaccttcttctggagtaatcctgctgggaaggagagatttttagtgacatgttcagtggttgctgtagaatt
tttttaaaaaaacaacaatttaaaatctatttaagtccaccagtgcctccctccttcttctactcccacccctcccat
gtccccccattcctcaaatccatttttaagagaagcagactgactttgaaaaggaggcgctggggtttgaaacctcccgcgtgc
taatctccctgggccccctcccggggaatcctctctgcaatcctgcgagggtctaggcccccttaggaagcctccgctctct
ttttcccaacagacctgtcttcacccctgggctttgaaagccagacaaagcagctgccccctctcctgccaaaggaggatcat
ccccaaaaagacagagggggagcccaagcccaagtccttctccagcagcgtttcccccaactccttaattttatttctcc
gctagattttaacgtccagccttccctcagctgagtggggagggcatcctgcaaaagggaacagaagaggccaagtccccca
agccacggcccggggttaaggctagagctgctggggaggggctgctgttttactcaccaccagcttcgcctcccccatcc
tgggcgccccctcctcagcttagctgtcagctgtccatcacctctccccactttctcatttgtgcttttttctctcgtaatag
aaaagtggggagcgcgtggggagccacccattcatcccgatatttccccctctcataacttctccccatccaggaggagttc
tcaggcctgggggtggggccccgggtgggtgcgggggagattcaacctgtgtgctgcgaaggacgagacttctcttgaacagtg
tgctgttgtaaacatatttgaaaactattaccaataaagttttgtttaaaaaaa
```

Step 2: Reading in the translation table

After constructing a word by recognizing the alphabet and _, the key and value values are created based on :

All unnecessary spaces and unnecessary characters such as ' are removed.

In [205...

```
Sehyoun_Jang(1)
dic = {}
with open("DNA_Amino_Acid_Translation_Table.txt", "r" ) as file2:
    alpha_list = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ_'
    for line in file2:
        a = line.rstrip().split(",")
        for x in a:
            temp_A = ""
            temp_B = ""
            sw = 1
            for char in x:
                if sw == 1:
                    if char in alpha_list:
```

```

        temp_A += char
    elif char == ":":
        sw = 2
    if sw == 2:
        if char in alpha_list:
            temp_B += char
    if temp_A != "" or temp_B != "":
        dic[temp_A] = temp_B

print(dic)

```

Name: Sehyoun Jang

sej324@lehigh.edu

Assignment ENGR10-S23-EN 7-3, section: 1

GROUP 3

```

-----
{'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M', 'ACA': 'T', 'ACC': 'T', 'ACG': 'T',
'ACT': 'T', 'AAC': 'N', 'AAT': 'N', 'AAA': 'K', 'AAG': 'K', 'AGC': 'S', 'AGT': 'S',
'AGA': 'R', 'AGG': 'R', 'CTA': 'L', 'CTC': 'L', 'CTG': 'L', 'CTT': 'L', 'CCA': 'P',
'CCC': 'P', 'CCG': 'P', 'CCT': 'P', 'CAC': 'H', 'CAT': 'H', 'CAA': 'Q', 'CAG': 'Q',
'CGA': 'R', 'CGC': 'R', 'CGG': 'R', 'CGT': 'R', 'GTA': 'V', 'GTC': 'V', 'GTG': 'V',
'GTT': 'V', 'GCA': 'A', 'GCC': 'A', 'GCG': 'A', 'GCT': 'A', 'GAC': 'D', 'GAT': 'D',
'GAA': 'E', 'GAG': 'E', 'GGA': 'G', 'GGC': 'G', 'GGG': 'G', 'GGT': 'G', 'TCA': 'S',
'TCC': 'S', 'TCG': 'S', 'TCT': 'S', 'TTC': 'F', 'TTT': 'F', 'TTA': 'L', 'TTG': 'L',
'TAC': 'Y', 'TAT': 'Y', 'TAA': '_', 'TAG': '_', 'TGC': 'C', 'TGT': 'C', 'TGA': '_',
'TGG': 'W'}

```

Step 3: Create the Amino Acid Sequence from the DNA sequence

Since the three alphabets produce one result value, the value is obtained by inserting the key value into the dictionary accordingly.

In [206...

```

Sehyoun_Jang(1)
DNA1 = DNA_read_function("DNA_1.txt")

def translate_DNA(word):
    result = ""
    for i in range(0, len(word)//3):
        temp = dic[word[i*3:i*3+3].upper()]
        result += temp
    return result

print(translate_DNA(DNA1))

```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

```
-----
TVVPVP_EQD_VMASNSSDVKEFLAKAKEDFLKKWESPAQNTAHLQFERIKTLGTGSFGRVMLVKHKETGNHYAMKILDKQKV
VKLKQIEHTLNEKRILQAVNFPFLVKLEFSFKDNSNLYMMEYVPGGEMFSLRRIGRFSEPHARFYAAQIVLTFEYLHSLDLI
YRDLKPENLLIDQQGYIQVTDGFAKRVKGRWTWLCGTPEYLAPEIILSKGYNKAVDWWALGVLIYEMAAGYPPFFADQPIQIY
EKIVSGKVRFPSSHFSDDLKDLLRNLLQVDLTKRFGNLKNGVNDIKNHKWFATTDWIAIYQRKVEAPFIPKFKGPGDTSNFDDYE
EEEIRVSINEKCGKEFSEF_GHACAPMGFLSFFFLVGGVGGD_TARGPQSSHLISPPHPPLWGGAGSPDNQDRNTSCS
PSSPSPSCPLSHFSLPLSPQPPSPSALPAHFCLF_TSFTSTPVRPGLAGVSRDRVWKEGLTLNSSPHPHPHPTQPQAPLAKGK_T
KRQPSFRSNPAWEGEIFSDMFSGLLARIFLKKQQFKILFKFHQCLPPSFYLSPSHVPPFLKSILKRSRLTLEREALGFEPFRC
_SPLGPSPGNPLCQSCGLGPFRKPPLSFSPDTLSSPLGFESQTKQLPLSLPKRSHPPKRQRGSPKPKSFLPAAFPPNSLILFS
ARF_RPAFPQLSGEGIPAKGNRRGQVPPSHGPGFKARAAGEGLPVLLTHQLPPPPSWAPLLQLSCQLSITSPPLSHLCFFLS__
KSGEPLGSHPIHPRISPSHNFSQEEFSGLGWGPWVRGRFNLCAAKDETSS_TVCCCKHI_KLLPIKFCLKK
```

Step 4: Create the Official Amino Acid Sequence

This part registers all alphabets as a list as one element except for line breaks.

```
In [207... Sehyoun_Jang(1)
def Amino_read_function(filename):
    result = ""
    with open(filename, 'r') as file:
        for i in file:
            for char in i.rstrip():
                result += char
    return result

print(Amino_read_function("Amino_Acid_1_37_1065.txt"))
Amino_1 = Amino_read_function("Amino_Acid_1_37_1065.txt")
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

```
-----
MASNSSDVKEFLAKAKEDFLKKWESPAQNTAHLQFERIKTLGTGSFGRVMLVKHKETGNHYAMKILDKQKVVKLKQIEHTLNE
KRILQAVNFPFLVKLEFSFKDNSNLYMMEYVPGGEMFSLRRIGRFSEPHARFYAAQIVLTFEYLHSLDLIYRDLKPENLLID
QQGYIQVTDGFAKRVKGRWTWLCGTPEYLAPEIILSKGYNKAVDWWALGVLIYEMAAGYPPFFADQPIQIYEKIVSGKVRFPS
HFSSDLKDLLRNLLQVDLTKRFGNLKNGVNDIKNHKWFATTDWIAIYQRKVEAPFIPKFKGPGDTSNFDDYEEEEIRVSINEK
GKEFSEF
```

```
In [208... print(Amino_read_function("Amino_Acid_3_107_427.txt"))
x = "MALWIRSLPLLALLVFSGPGTSYAAANQHLCGSHLVEALYLVCGERGFFYSKARRDVEQPLVSSPLRGEAGVLPFQQ
MALWIRSLPLLALLVFSGPGTSYAAANQHLCGSHLVEALYLVCGERGFFYSKARRDVEQPLVSSPL
RGEAGVLPFQQEYEEKVKGIVEQCCHNTCSLYQLENYCN
```

Step 5: Compare two Amino Acid Sequences

It can be solved by slicing one large string and compare it. In the case of DNA, don't forget to divide it by three. Since three alphabets have been converted

```
In [209... Sehyoun_Jang(1)
def compare_function(DNA, Amino, start_num, end_num):
    if DNA[(start_num-1)//3:end_num//3] == Amino:
        return True
    else:
        return False

Amino_1 = Amino_read_function("Amino_Acid_1_37_1065.txt")
DNA1 = DNA_read_function("DNA_1.txt")
DNA1_traslated = translate_DNA(DNA1)

print(compare_function(DNA1_traslated, Amino_1, 37, 1065))
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

 True

Step 6: Confirm the three test DNA sequences

Just rename the files to the functions that have already been completed.

```
In [210... Sehyoun_Jang(1)
Amino_1 = Amino_read_function("Amino_Acid_1_37_1065.txt")
DNA1 = DNA_read_function("DNA_1.txt")
DNA1_traslated = translate_DNA(DNA1)
print(compare_function(DNA1_traslated, Amino_1, 37, 1065))
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

 True

```
In [211... Sehyoun_Jang(1)
Amino_2 = Amino_read_function("Amino_Acid_2_10_1140.txt")
DNA2 = DNA_read_function("DNA_2.txt")
DNA2_traslated = translate_DNA(DNA2)
print(compare_function(DNA2_traslated, Amino_2, 10, 1140))
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

 True

```
In [212... Sehyoun_Jang(1)
Amino_3 = Amino_read_function("Amino_Acid_3_107_427.txt")
DNA3 = DNA_read_function("DNA_3.txt")
DNA3_traslated = translate_DNA(DNA3)
print(compare_function(DNA3_traslated, Amino_3, 107, 427))
```

```
# I honestly don't know this part.
# MALWIRSLPLLALLVFSGPGTSYAAANQHLCGSHLVEALYLVCGL
# Is there a space on purpose in this content or should I remove this? Is this the
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 1
 GROUP 3

 False

Section 2: Mutated DNA

It interprets both original and mutated files using existing functions. After that, use index numbers to compare characters one by one and add them to the dictionary if they are wrong.

```
In [213... Sehyoun_Jang(2)
def compare_mutated_data(original_data, mutated_data):
    dic = {}
    original = DNA_read_function(original_data)
    original_traslated = translate_DNA(original)
    mutated = DNA_read_function(mutated_data)
    mutated_traslated = translate_DNA(mutated)

    for i in range (len(original_traslated)):
        if original_traslated[i] != mutated_traslated[i]:
            loc = i # key
            original_char = original_traslated[i]
            mutated_char = mutated_traslated[i]
            value = mutated_char + " instead of " + original_char
            dic[loc] = value
    return dic

print(compare_mutated_data("DNA_1.txt", "DNA_1_mutated.txt"))
print(compare_mutated_data("DNA_2.txt", "DNA_2_mutated.txt"))
print(compare_mutated_data("DNA_3.txt", "DNA_3_mutated.txt"))
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 2
 GROUP 3

 {332: 'Y instead of D'}
 {131: 'R instead of Q', 142: 'V instead of L', 250: 'L instead of Q'}
 {48: 'T instead of P', 102: 'S instead of A'}

If there is a difference from the existing function, this time returning the dictionary.
 (Previously, only True False was returned) Make some modifications to the existing function to return the dictionary.

In [214...

Sehyoun_Jang(2)

```
def compare_function2(Mutated_DNA, Amino, start_num, end_num):
    Mutated_DNA = Mutated_DNA[(start_num-1)//3:end_num//3]
    dic = {}
    for i in range (len(Mutated_DNA)):
        if Mutated_DNA[i] != Amino[i]:
            loc = i # key
            original_char = Mutated_DNA[i]
            mutated_char = Amino[i]
            value = mutated_char + " instead of " + original_char
            dic[loc] = value

    return dic

Amino_1 = Amino_read_function("Amino_Acid_1_37_1065.txt")
mutated_1 = DNA_read_function("DNA_1_mutated.txt")
mutated_traslated_1 = translate_DNA(mutated_1)
print(compare_function2(mutated_traslated_1, Amino_1, 37, 1065))

Amino_2 = Amino_read_function("Amino_Acid_2_10_1140.txt")
mutated_2 = DNA_read_function("DNA_2_mutated.txt")
mutated_traslated_2 = translate_DNA(mutated_2)
print(compare_function2(mutated_traslated_2, Amino_2, 10, 1140))

Amino_3 = Amino_read_function("Amino_Acid_3_107_427.txt")
mutated_3 = DNA_read_function("DNA_3_mutated.txt")
mutated_traslated_3 = translate_DNA(mutated_3)
print(compare_function2(mutated_traslated_3, Amino_3, 107, 427))
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 2
 GROUP 3

```
-----
{320: 'D instead of Y'}
{128: 'Q instead of R', 139: 'L instead of V', 247: 'Q instead of L'}
{0: 'M instead of H', 1: 'A instead of G', 2: 'L instead of S', 3: 'W instead of L',
4: 'I instead of D', 5: 'R instead of P', 6: 'S instead of I', 7: 'L instead of T',
8: 'P instead of A', 9: 'L instead of S', 10: 'L instead of S', 11: 'A instead of
G', 12: 'L instead of S', 13: 'L instead of T', 14: 'V instead of C', 15: 'F instead
of L', 16: 'S instead of F', 17: 'G instead of W', 19: 'G instead of W', 20: 'T inst
ead of N', 21: 'S instead of Q', 22: 'Y instead of L', 23: 'A instead of C', 24: 'A
instead of S', 25: 'A instead of C', 26: 'N instead of Q', 27: 'Q instead of P', 28:
'H instead of A', 29: 'L instead of P', 30: 'C instead of L', 31: 'G instead of W',
32: 'S instead of L', 33: 'H instead of P', 35: 'V instead of G', 36: 'E instead of
G', 37: 'A instead of G', 38: 'L instead of S', 39: 'Y instead of L', 40: 'L instead
of P', 41: 'V instead of G', 42: 'C instead of V', 43: 'G instead of W', 44: ' inst
ead of R', 45: ' instead of A', 46: ' instead of W', 47: ' instead of L', 48: '
instead of L', 49: ' instead of L', 50: ' instead of L', 51: ' instead of P', 52:
' instead of Q', 53: ' instead of S', 54: ' instead of P', 55: ' instead of T',
56: ' instead of G', 57: ' instead of C', 58: ' instead of R', 59: ' instead of
A', 60: ' instead of A', 61: 'E instead of P', 62: 'R instead of S', 63: 'G instead
of E', 64: 'F instead of Q', 65: 'F instead of S', 66: 'Y instead of L', 68: 'P inst
ead of W', 69: 'K instead of R', 70: 'A instead of G', 72: 'R instead of S', 73: 'D
instead of A', 74: 'V instead of A', 75: 'E instead of F', 76: 'Q instead of P', 77:
'P instead of A', 78: 'L instead of G', 79: 'V instead of G', 80: 'S instead of I',
81: 'S instead of R', 82: 'P instead of E', 83: 'L instead of S', 84: 'R instead of
Q', 85: 'G instead of A', 86: 'E instead of R', 87: 'A instead of D', 88: 'G instead
of C', 89: 'V instead of _', 90: 'L instead of A', 91: 'P instead of M', 92: 'F inst
ead of L', 93: 'Q instead of P', 94: 'Q instead of _', 95: 'E instead of H', 96: 'E
instead of V', 97: 'Y instead of F', 98: 'E instead of P', 99: 'K instead of L', 10
0: 'V instead of P', 101: 'K instead of T', 102: 'R instead of G', 103: 'G instead o
f E', 104: 'I instead of L', 105: 'V instead of L', 106: 'E instead of Q'}
```

Section 3: Analysis via Pandas

I've done quite a lot of practice dealing with Kaggle's vast amount of data in Pandas before.
 The instructions are easy and kind, so I can do it step by step.

```
In [215... #1 Print out the overall dimensions of the dataset (i.e., How many rows and columns
import pandas as pd
import matplotlib.pyplot as plt

Sehyoun_Jang(3)

codon_df = pd.read_csv('codon_usage.csv')
print("Dimensions of dataset:", codon_df.shape, end = "(rows and columns)")
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 3
 GROUP 3

 Dimensions of dataset: (13026, 69)(rows and columns)

In [216... Sehyoun_Jang(3)
2 Display only the weights for the codons CUU, CUC, CUA and CUG for the first five
 print(codon_df.loc[:,4, ['CUU', 'CUC', 'CUA', 'CUG']])

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 3
 GROUP 3

	CUU	CUC	CUA	CUG
0	0.01203	0.03208	0.00100	0.04010
1	0.00407	0.02849	0.00204	0.04410
2	0.00782	0.01111	0.01028	0.01193
3	0.01567	0.01358	0.00940	0.01723
4	0.01380	0.00548	0.00473	0.02076

In [217... *# 3 Filter the dataset to include just those species containing "Tomato" then print*
 Sehyoun_Jang(3)
 tomato_df = codon_df[codon_df['SpeciesName'].str.contains('Tomato')]
 print(tomato_df)

Name: Sehyoun Jang

sej324@lehigh.edu

Assignment ENGR10-S23-EN 7-3, section: 3

GROUP 3

```
-----
      Kingdom  DNAType  SpeciesID  Ncodons  \
456      vr1      0      12281    4478
470      vr1      0      12315    12338
480      vr1      0      123735   3424
522      vr1      0      12851    2261
565      vr1      0      138314   17982
...      ...      ...      ...      ...
2695     vr1      0      71186    2817
2791     vr1      0      85752   11883
2811     vr1      0      92963    3923
2826     vr1      0      96054    1104
5420     bct      0      434404   1737
```

```

                                SpeciesName    UUU    UUC    UUA  \
456                        Tomato ringspot virus 0.03283 0.01362 0.01742
470                        Tomato aspermy virus  0.01759 0.01597 0.01662
480  Tomato yellow leaf curl Sardinia virus 0.02541 0.02132 0.02044
522                        Tomato chlorotic spot virus 0.02521 0.01725 0.01504
565                        Tomato mosaic virus  0.02452 0.02085 0.02135
...                        ...                ...    ...    ...
2695      Tomato leaf curl Sinaloa virus 0.02201 0.02130 0.00852
2791  Tomato yellow leaf curl Thailand virus 0.02264 0.02104 0.01464
2811      Tomato severe leaf curl virus  0.01886 0.02422 0.01912
2826      Tomato leaf curl Philippines virus 0.02355 0.01721 0.01630
5420      Tomato big bud phytoplasma  0.06793 0.00979 0.07254
```

```

      UUG    CUU    ...    CGG    AGA    AGG    GAU    GAC  \
456  0.03059 0.02144 ... 0.00491 0.00759 0.01675 0.03171 0.01117
470  0.01289 0.02237 ... 0.00251 0.01086 0.00681 0.03599 0.01986
480  0.01110 0.01373 ... 0.00467 0.01373 0.02015 0.03183 0.02132
522  0.01681 0.01725 ... 0.00177 0.01636 0.00708 0.04113 0.01106
565  0.02358 0.01741 ... 0.00384 0.02197 0.01468 0.03787 0.02035
...   ...    ...    ...    ...    ...    ...    ...
2695 0.01029 0.00958 ... 0.01065 0.01917 0.01704 0.03266 0.02272
2791 0.01044 0.01203 ... 0.00572 0.01708 0.01683 0.02651 0.01515
2811 0.00663 0.00612 ... 0.00892 0.01300 0.02218 0.03059 0.01759
2826 0.01178 0.01449 ... 0.00453 0.01993 0.01902 0.03714 0.01540
5420 0.01094 0.01324 ... 0.00288 0.00633 0.00000 0.03627 0.00345
```

```

      GAA    GAG    UAA    UAG    UGA
456  0.02479 0.02858 0.00067 0.00022 0.00000
470  0.02658 0.01824 0.00008 0.00000 0.00332
480  0.02891 0.01431 0.00292 0.00146 0.00117
522  0.04556 0.01681 0.00088 0.00000 0.00133
565  0.03537 0.02653 0.00156 0.00056 0.00000
...   ...    ...    ...    ...
2695 0.02130 0.02094 0.00248 0.00071 0.00106
2791 0.02777 0.01843 0.00269 0.00118 0.00151
2811 0.01886 0.02447 0.00408 0.00102 0.00000
2826 0.01902 0.02264 0.00362 0.00091 0.00091
5420 0.04491 0.00518 0.00230 0.00115 0.00058
```

[156 rows x 69 columns]

In [218... *#4 Find the species that has the largest weight of AUA then print out its data*

```
Sehyoun_Jang(3)
AUA_max = codon_df['AUA'].max()
max_species = codon_df.loc[codon_df['AUA'] == AUA_max]
print(max_species)
```

Name: Sehyoun Jang

sej324@lehigh.edu

Assignment ENGR10-S23-EN 7-3, section: 3

GROUP 3

```
-----
      Kingdom  DNAtype  SpeciesID  Ncodons  \
8699      inv         1    124230    4680

                                     SpeciesName      UUU  UUC      UUA      UUG  \
8699  mitochondrion Maoricicada campbelli  0.11538  0.0    0.12158  0.00043

      CUU  ...  CGG      AGA  AGG      GAU  GAC  GAA  GAG      UAA  UAG  \
8699  0.0  ...  0.0  0.01923  0.0  0.0235  0.0  0.0  0.0  0.01923  0.0

      UGA
8699  0.05769
```

[1 rows x 69 columns]

In [219... *#5 Filter the dataset to include only those species with a GUC weight larger than .*

```
Sehyoun_Jang(3)
filtered_dataset = codon_df[codon_df['GUC'] > 0.06]
print(filtered_dataset)
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 3
 GROUP 3

```
-----
      Kingdom  DNAType  SpeciesID  Ncodons  SpeciesName \
3826      bct       0      159599    2038      Streptomyces sp. NRRL 5331
3901      bct       0       1708    3386      Cellulomonas fimi
4249      bct       0      219305    2380      Micromonospora sp. ATCC 39149
4280      arc       0       2246    21750      Haloferax volcanii
4371      arc       0      242927    1207      Haloarcula sp. D1
4489      bct       0      266940  1494419  Kineococcus radiotolerans SRS30216
5912      arc       0       79680    1329      Thermococcus sp. KS-8
7832      pln       0       4580    2209      Zea perennis
9200      inv       0      38949    2182      Perna canaliculus
9206      inv       0      39564    1772      Pseudomicrothorax dubius
9423      inv       0       5872    7829      Babesia equi
```

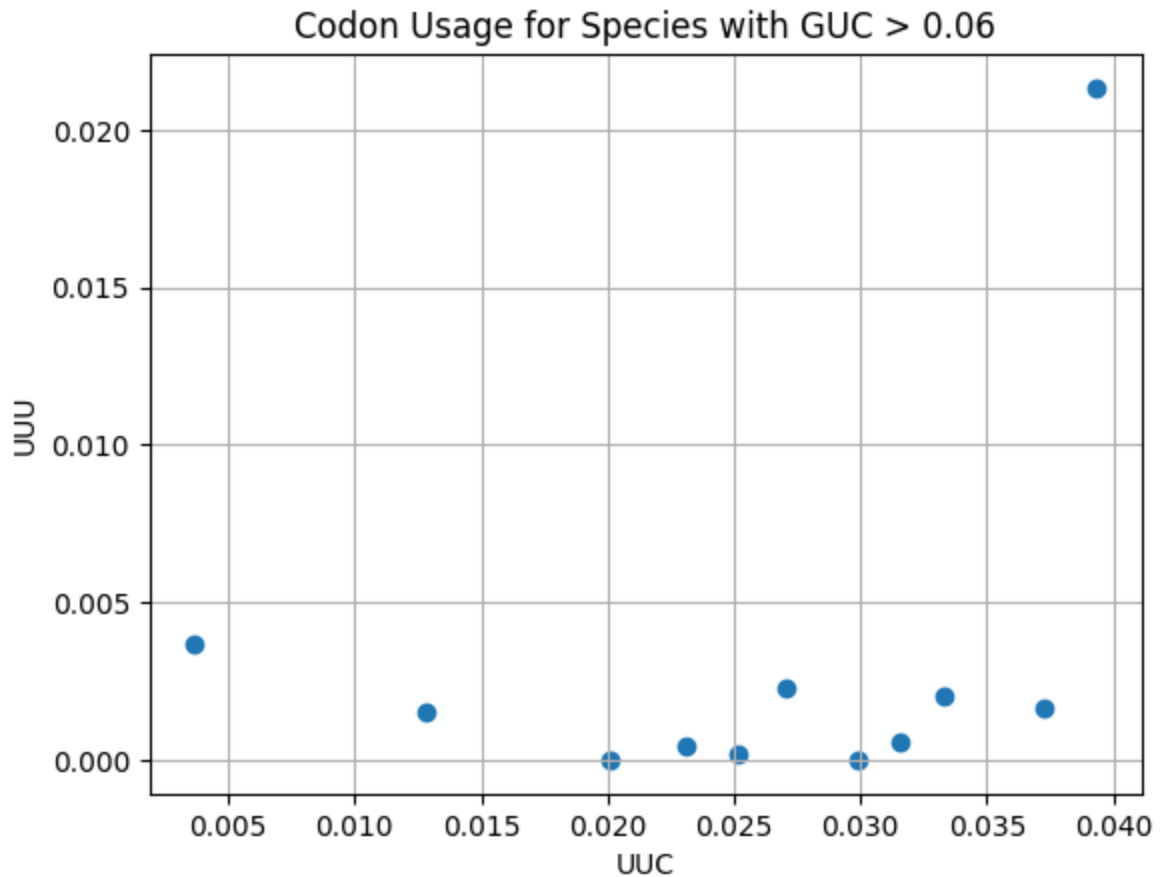
```
      UUU      UUC      UUA      UUG      CUU  ...      CGG      AGA  \
3826  0.00000  0.02012  0.00049  0.00196  0.00196  ...  0.02110  0.00098
3901  0.00059  0.03160  0.00000  0.00059  0.00089  ...  0.02126  0.00000
4249  0.00042  0.02311  0.00000  0.00252  0.00210  ...  0.02395  0.00000
4280  0.00202  0.03333  0.00018  0.00262  0.00345  ...  0.01526  0.00120
4371  0.00166  0.03728  0.00083  0.00663  0.00166  ...  0.01574  0.00083
4489  0.00020  0.02516  0.00005  0.00282  0.00067  ...  0.02801  0.00032
5912  0.00150  0.01279  0.00150  0.00150  0.01354  ...  0.00226  0.00602
7832  0.00000  0.02988  0.00000  0.00453  0.00362  ...  0.00000  0.00000
9200  0.00367  0.00367  0.00367  0.00550  0.00779  ...  0.00046  0.00458
9206  0.00226  0.02709  0.00056  0.00734  0.00451  ...  0.00113  0.01749
9423  0.02133  0.03934  0.00077  0.02631  0.00971  ...  0.00000  0.01341
```

```
      AGG      GAU      GAC      GAA      GAG      UAA      UAG      UGA
3826  0.00147  0.00294  0.05839  0.00687  0.05250  0.00098  0.00000  0.00196
3901  0.00118  0.00030  0.07649  0.00059  0.04578  0.00000  0.00000  0.00148
4249  0.00126  0.00126  0.05420  0.00210  0.06429  0.00042  0.00042  0.00378
4280  0.00069  0.00262  0.08671  0.02446  0.06184  0.00069  0.00041  0.00129
4371  0.00083  0.00829  0.08451  0.02486  0.07291  0.00083  0.00000  0.00249
4489  0.00171  0.00195  0.05818  0.00527  0.04831  0.00004  0.00025  0.00284
5912  0.03160  0.01129  0.05643  0.00978  0.08804  0.00075  0.00000  0.00150
7832  0.00045  0.00724  0.05613  0.00770  0.02626  0.00000  0.00453  0.00724
9200  0.00092  0.01650  0.01421  0.00917  0.00458  0.00000  0.00183  0.00046
9206  0.00000  0.00959  0.02144  0.00395  0.04628  0.00056  0.00113  0.00000
9423  0.00575  0.02682  0.04458  0.03308  0.06361  0.00332  0.00000  0.00000
```

[11 rows x 69 columns]

```
In [220... #6 For the species with GUC larger than .06, plot the UUU weight as the y-axis and
Sehyoun_Jang(3)
plt.scatter(filtered_dataset['UUC'], filtered_dataset['UUU'])
plt.title('Codon Usage for Species with GUC > 0.06')
plt.xlabel('UUC')
plt.ylabel('UUU')
plt.grid()
plt.show()
```

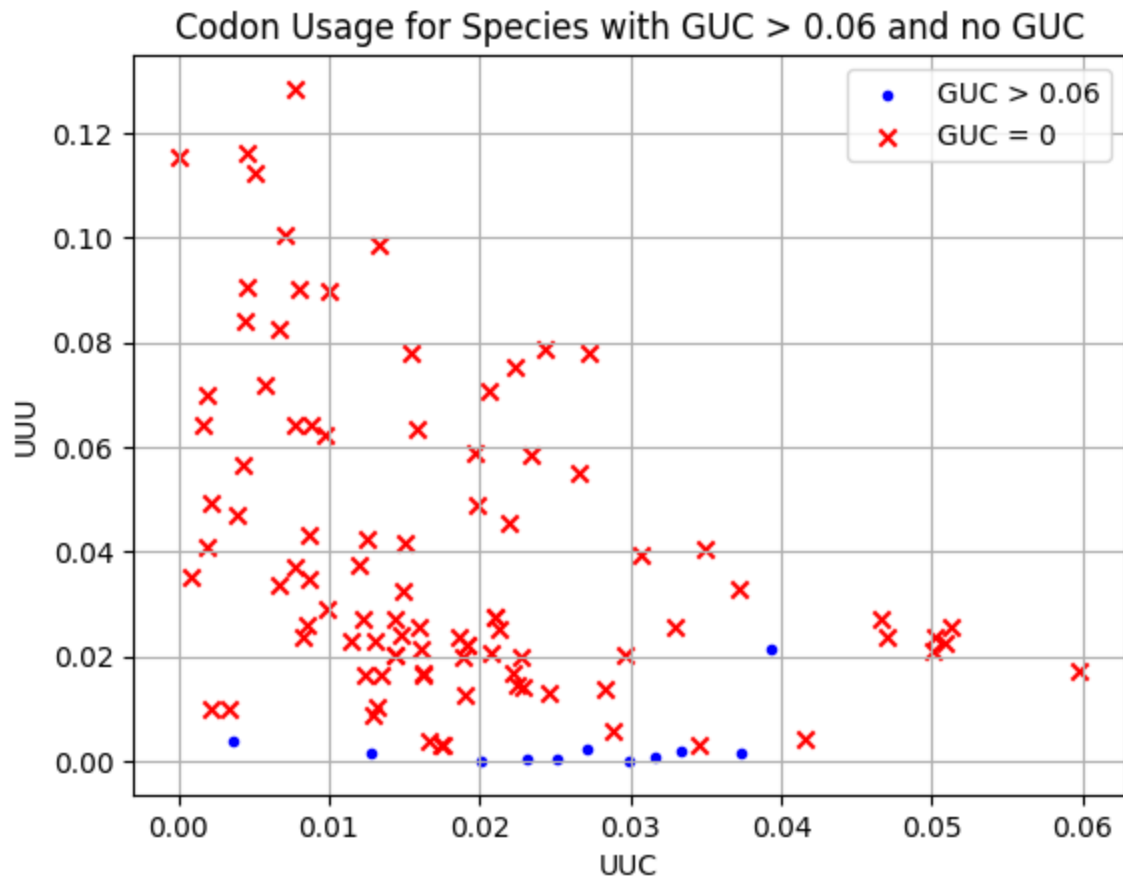
Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 3
 GROUP 3



In [221...

```
#7 On the same plot, plot the UUU and UUC weights for the species with no GUC, using
Sehyoun_Jang(3)
no_GUC_df = codon_df[codon_df['GUC'] == 0]
plt.scatter(filtered_dataset['UUC'], filtered_dataset['UUU'], color='blue', marker='.')
plt.scatter(no_GUC_df['UUC'], no_GUC_df['UUU'], color='red', marker='x')
plt.title('Codon Usage for Species with GUC > 0.06 and no GUC')
plt.xlabel('UUC')
plt.ylabel('UUU')
plt.legend(['GUC > 0.06', 'GUC = 0'])
plt.grid()
plt.show()
```

Name: Sehyoun Jang
 sej324@lehigh.edu
 Assignment ENGR10-S23-EN 7-3, section: 3
 GROUP 3



Section 1 and Section 2 of this assignment were interesting. Actually, I'm not sure I did it right. I don't know if I understood the instruction correctly. Section 3 was irrelevant to the previous section.